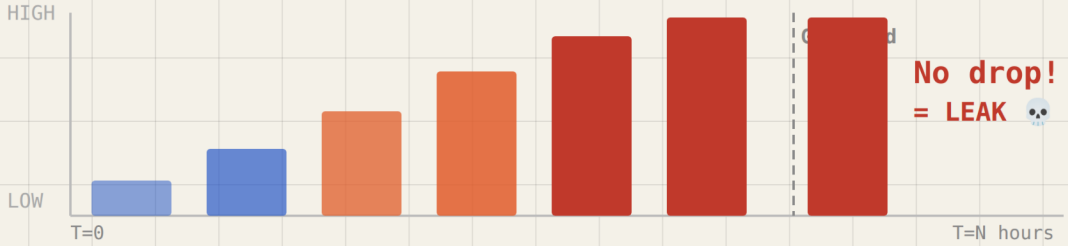
JAVA + SPRING BOOT

MEMORY LEAKS

HOW TO DEBUG THEM

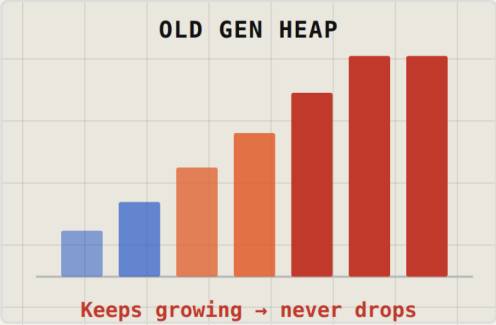
→ Spring Boot

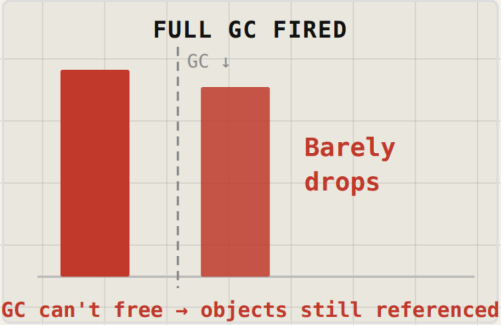
HEAP USAGE OVER TIME



HOW TO SPOT IT

DETECTING THE LEAK





KEY SIGNALS

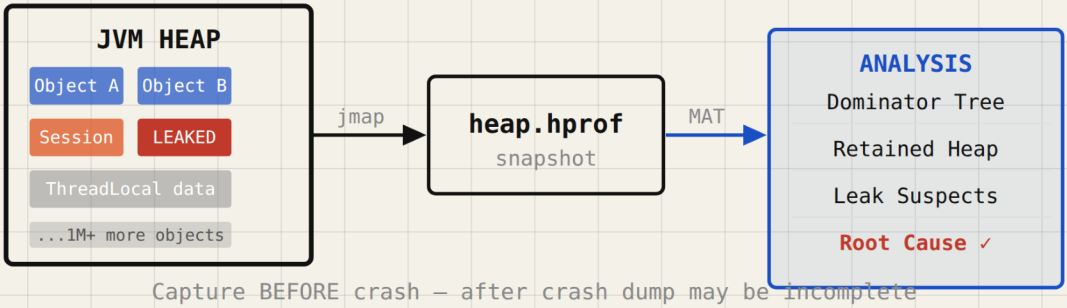
Old Gen heap grows continuously. Frequent Full GC fires but memory does not recover. Heap usage never plateaus — always climbing.

TOOLS TO MONITOR

① Spring Actuator — /actuator/metrics/jvm.memory.used
② VisualVM or JConsole — observe heap in real time
③ Trigger GC manually and watch if memory drops

CAPTURE THE EVIDENCE

HEAP DUMP



Capture BEFORE crash — after crash dump may be incomplete

CAPTURE COMMAND

```
jmap -dump:live,format=b,file=heap.hprof <pid>
```

Or auto-capture on OOM: -XX:+HeapDumpOnOutOfMemoryError

ANALYZE WITH ECLIPSE MAT

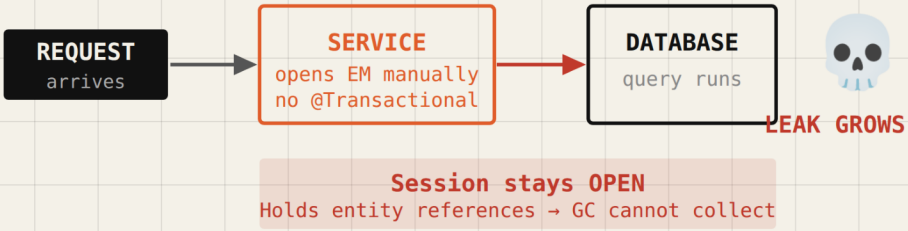
Dominator Tree — which objects hold the most memory

Retained Heap — what gets freed if object is removed

Leak Suspects Report — MAT auto-highlights patterns

MOST COMMON SPRING LEAK

UNCLOSED ENTITYMANAGER



WHY IT LEAKS

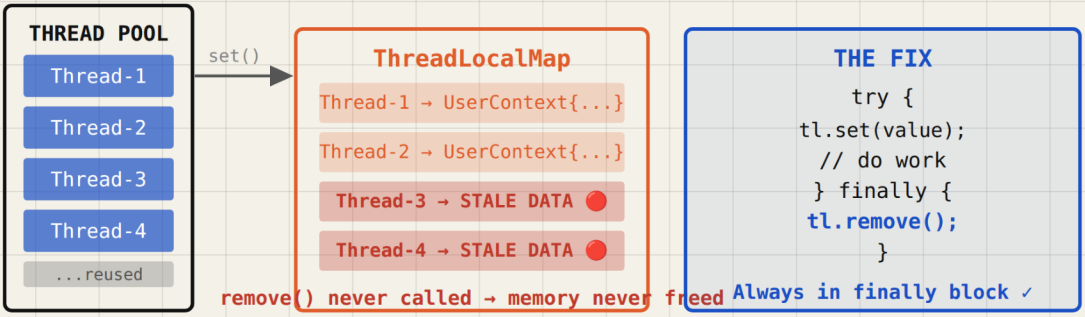
Manual EntityManager not closed → DB sessions pile up → each holds strong references to entity objects → GC blocked from freeing memory.

FIX

① Use @Transactional — Spring manages session lifecycle
② Never manage EntityManager manually in service layer
③ Use Spring Data JPA repositories — sessions auto-closed

THREAD POOL TRAP

THREADLOCAL NOT CLEARED



WHY IT LEAKS

Threads in pool are reused — never destroyed. Without remove(), old request data stays in ThreadLocalMap forever. Object graph cannot be GC'd.

FIX

Always call threadLocal.remove() in a finally block after request completes. Never rely on the thread ending — it won't in a thread pool.

HASHMAP TRAP

CACHE WITHOUT EVICTION

HashMap / ConcurrentHashMap

No TTL. No max size.

```
entry[0001] → ProductData{...}
entry[0002] → ProductData{...}
entry[0003] → ProductData{...}
entry[N] → keeps growing...
```

Heap dump: 2GB in this Map 💀

swap →

Caffeine Cache (fix)

```
maximumSize(10_000)
expireAfterWrite(30, MINUTES)
expireAfterAccess(10, MINUTES)
```

Bounded. Evicts. Memory stays stable ✓

WHY IT LEAKS

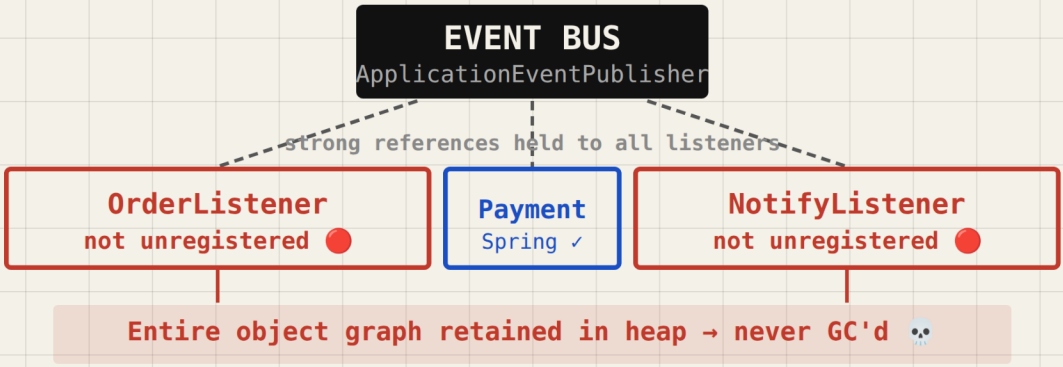
HashMap holds strong references. Without a size limit or TTL, entries accumulate forever. Heap dump will show one gigantic Map as the dominator.

FIX

① Replace HashMap with Caffeine — configure max size + TTL
② For distributed systems, use Redis with expiry policies
③ Never use a raw Map as a production cache

SILENT OBJECT RETAINER

LISTENER NOT UNREGISTERED



WHY IT LEAKS

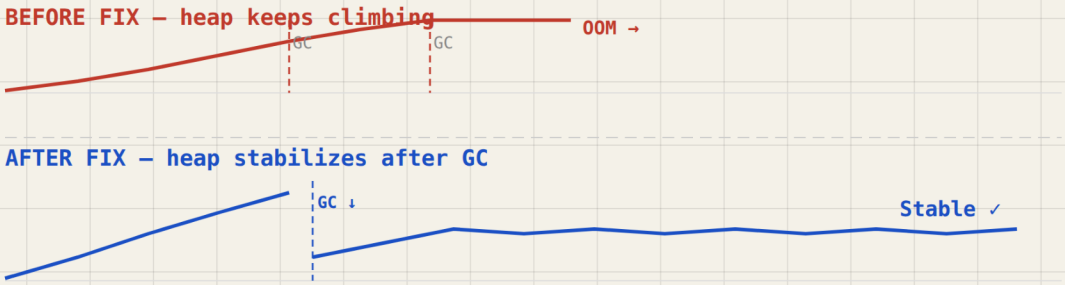
Event bus holds a strong reference to each registered listener. If listener is not unregistered, neither it nor its entire object graph can ever be garbage collected.

FIX

① Prefer Spring @EventListener — framework manages lifecycle
② For manual listeners: unregister in destroy / @PreDestroy
③ Use weak references if listener lifecycle is unclear

AFTER FIXING — VERIFY

FINAL VALIDATION



Run load test → monitor → verify plateau after GC

VALIDATION STEPS

① Run a full load test at production-like traffic
② Monitor /actuator/metrics/jvm.memory.used over time
③ Trigger GC manually — verify heap actually drops
④ Confirm no continuous growth pattern over 30+ min

Key insight: A memory leak is not a JVM failure. It's always an unreleased reference. The goal is to find what is preventing GC and remove that reference.

6 MEMORY LEAKS

UNCLOSED ENTITYMANAGER

Sessions stay open → entity refs block GC

→ Use @Transactional, never manual EM

THREADLOCAL NOT CLEARED

Reused threads hold stale data forever

→ Always call remove() in finally block

CACHE WITHOUT EVICTION

HashMap grows indefinitely → OOM

→ Use Caffeine with max size + TTL

LISTENERS NOT UNREGISTERED

Event bus holds object graph in memory

→ Use @EventListener or unregister in @PreDestroy

STATIC COLLECTIONS

GC roots — never collected by JVM

→ Avoid static for dynamic data, add limits

SINGLETON BEAN HOLDING DATA

Large data in singleton = permanent heap usage

→ Stream data, externalize storage